

Image Signal Processor (ISP)

Core

Team Name: RTL ARCHITECTS

Team Members:

- Royce Niran George A
- Angelin Jebisha C
- Nithiyasri H
- Ranjith Ganesh B.

College : Saveetha Engineering College

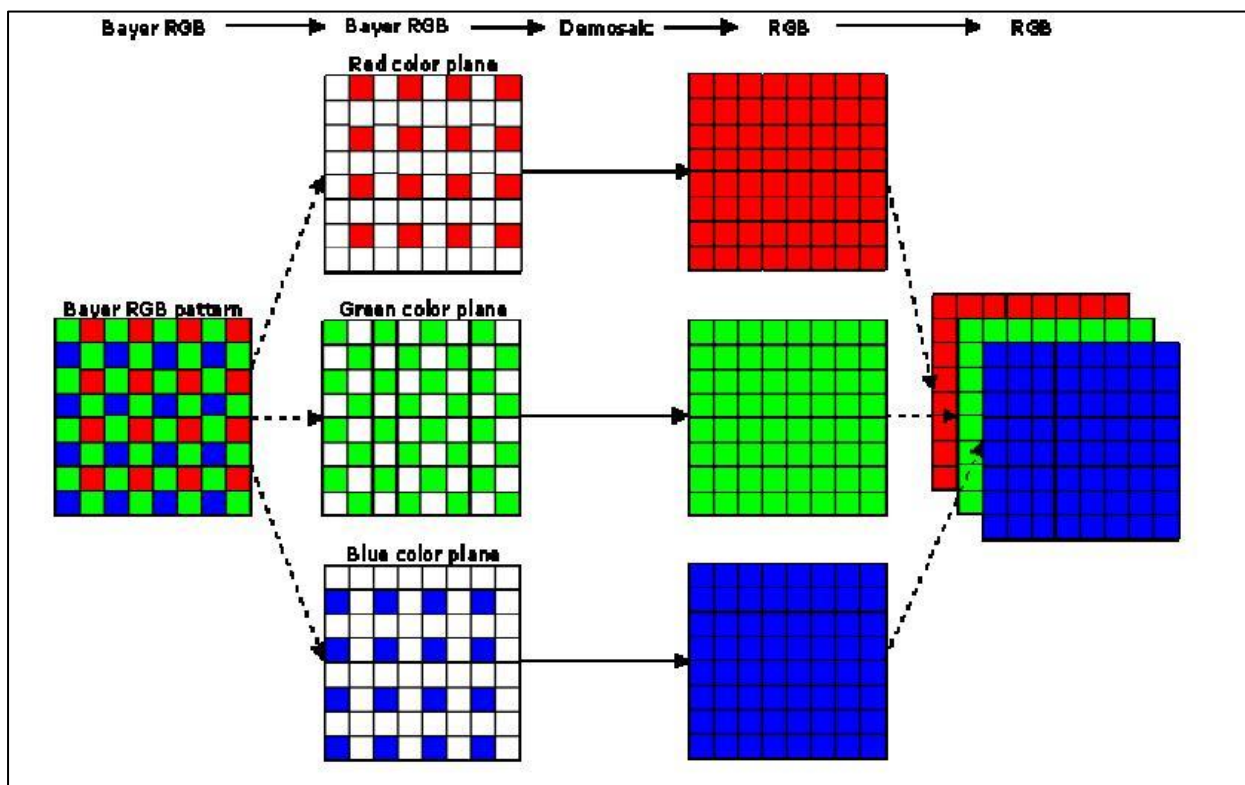
Theme II: Vision & Imaging IP Development

Image Signal Processor (ISP) Core

1. Abstract

This report presents the architectural design, hardware implementation, and system-level validation of a high-performance **Image Signal Processor (ISP) Core** optimized for FPGA-based acceleration. In modern embedded vision systems—particularly in autonomous vehicles, robotics, and real-time surveillance—**vision pipeline latency** and **throughput efficiency** are critical determinants of system performance.

The proposed ISP core addresses the computational bottlenecks associated with reconstructing full-color images from raw sensor data. Image sensors inherently capture monochromatic intensity values through a **Bayer Color Filter Array (CFA)**, necessitating computationally intensive interpolation techniques such as demosaicing to recover full RGB information. Software-based implementations on general-purpose processors introduce significant latency and limit real-time capability.



To overcome these limitations, this work introduces a **modular, synthesizable ISP IP core** that integrates:

- **Bilinear Demosaicing Engine** for color reconstruction
- **Digital Gain Control Unit** using fixed-point arithmetic
- **Streaming 3-Line Buffer Architecture** for efficient spatial data reuse
- **AXI4-Stream Interfaces** for high-throughput data movement
- **AXI-Lite Control Interface** for runtime configurability

The architecture leverages **fine-grained pipelining** and **data streaming techniques**, enabling continuous pixel processing at an initiation interval (II) of 1, thereby achieving **one pixel per clock cycle** without intermediate frame buffering.

The ISP core is deployed on a **Xilinx Zynq-7000 System-on-Chip (SoC)** using the PYNQ framework, integrating programmable logic (PL) with ARM-based processing system (PS). Experimental results demonstrate a processing latency of approximately **0.012 seconds per 720p frame**, corresponding to a throughput of **82.4 frames per second (FPS)**. This represents a **~13× performance improvement** compared to conventional CPU-based implementations (~6 FPS), while maintaining negligible processor utilization.

2. Introduction

2.1 Background and Problem Statement

Modern digital image sensors employ a **Color Filter Array (CFA)**—most commonly the **Bayer Pattern (RGGB)**—to capture visual information. Due to physical and cost constraints, each pixel in the sensor records **only a single color component** (Red, Green, or Blue), rather than full RGB data.

As a result, the raw output from the image sensor appears as a mosaiced, grayscale-like image in which each pixel carries only partial color information corresponding to a single color component (red, green, or blue). Consequently, the complete color representation of the scene cannot be obtained directly from individual pixels. Instead, the missing color components must be estimated by analyzing neighboring pixels through interpolation techniques, enabling the reconstruction of a full-

resolution RGB image. This reconstruction process is known as **demosaicing**, and it involves spatial interpolation over a local neighborhood, typically a: **3 X 3** window of pixels.

Most healthcare systems rely on cloud-based machine learning, where data is processed remotely. While computationally powerful, this approach suffers from latency, high bandwidth requirements, privacy concerns, and dependence on stable connectivity. These limitations make cloud-based solutions unsuitable for real-time healthcare applications, emphasizing the need for local processing.

2.2 Challenges in Software-Based ISP Pipelines

Performing demosaicing and associated ISP operations (e.g., gain control, filtering) on a traditional CPU introduces several critical limitations:

- **High Computational Complexity**
 - Each output pixel requires multiple arithmetic operations involving neighboring pixels
 - Results in significant processing overhead for high-resolution images
- **Real-Time Constraints**
 - For 720p resolution (~1 million pixels per frame) at 60 FPS:
 - ~60 million pixels must be processed per second
 - CPUs struggle to sustain this throughput without optimization
- **Power Inefficiency**
 - Continuous high-load computation increases power consumption
 - Unsuitable for **edge devices** (drones, robotics, embedded vision systems)
- **Memory Bottlenecks**
 - Frequent memory access for neighborhood pixels leads to cache inefficiencies.

2.3 Problem Statement

Design a **low-latency, high-throughput ISP core** capable of converting RAW Bayer data into full RGB images in real time, while minimizing CPU utilization and power consumption.

2.4 FPGA as an Edge AI Platform

Key Advantages of FPGA-Based ISP

- **Deep Pipelining**
 - Multiple processing stages operate **concurrently**
 - Each stage processes a different pixel at the same time
- **Spatial Parallelism**
 - Unlike CPUs (temporal execution), FPGAs execute operations in **parallel hardware blocks**
- **Deterministic Execution**
 - No operating system interference
 - Predictable latency and throughput
- **Low Latency Streaming**
 - Data flows continuously through the pipeline without buffering entire frames

Conceptual Comparison

Feature	CPU-Based Processing	FPGA-Based Processing
Execution Model	Sequential	Parallel (Spatial)
Throughput	Limited	High (1 pixel/clock)
Latency	High	Low
Determinism	Low (OS dependent)	High
Power Efficiency	Moderate–High	Low

2.5 Limitations of Existing Approaches

Despite advancements in software optimization and GPU acceleration, several limitations persist in conventional ISP implementations.

1. Sequential Bottleneck

Demosaicing inherently requires accessing neighboring pixels within a 3×3 window, leading to repeated memory access patterns. In a CPU-based implementation, this involves sequential steps of loading pixel data from memory, performing arithmetic computations, and storing the processed results back to memory. Such a workflow introduces significant overhead due to frequent memory transactions and limited parallelism.

Consequently, this results in increased processing latency, inefficient cache utilization, and poor scalability when handling higher-resolution images or real-time video streams.

2. Non-Deterministic Behavior

CPU execution is inherently non-deterministic, as it is influenced by factors such as operating system scheduling, interrupt handling, and concurrent background processes. These external interferences disrupt consistent execution timing, especially in compute-intensive tasks like image processing.

As a result, systems may experience frame drops, variations in processing time (jitter), and overall unreliable performance. This unpredictability is particularly problematic in real-time applications such as autonomous navigation and robotics control loops, where consistent and timely processing is critical.

3. Memory Bandwidth Constraints

- Frequent read/write operations for neighborhood access
- Bottleneck in systems with limited cache or memory bandwidth

4. Limited Scalability

- Increasing resolution (e.g., 1080p, 4K) leads to: Exponential growth in computational demand and Reduced achievable frame rates.

2.5 Motivation

The growing demand for **real-time intelligent vision systems** necessitates a paradigm shift from software-centric processing to **hardware-accelerated pipelines**.

To develop a “**Zero-Latency**” **Vision Front-End** capable of delivering high-quality RGB image streams directly to downstream AI inference engines.

Vision:

The proposed ISP core acts as a **front-end accelerator**, ensuring that:

- Raw sensor data is converted into usable RGB format **instantly**
- AI models receive **clean, processed input**
- The system maintains **low power consumption and deterministic performance**

3. Novelty and Technical Contributions

3.1 Streaming Line Buffer Architecture (Rolling Window)

A fundamental challenge in ISP design is the efficient realization of spatial operations such as demosaicing, which require access to neighboring pixels. Conventional approaches rely on **frame buffering**, introducing latency and memory overhead.

Proposed Approach: Rolling Window using BRAM

The proposed ISP core employs a **streaming 3-line buffer architecture**, implemented using FPGA **Block RAM (BRAM)** resources. Instead of storing entire frames, only **three rows of pixels** are maintained at any given time.

Key Design Features

- **Continuous Data Streaming**
 - Pixels are processed as they arrive (no frame-level storage)
- **3×3 Sliding Window Generation**
 - A local neighborhood is dynamically formed using:
 - 3 line buffers (rows)
 - Shift registers (columns)

- **Rolling Update Mechanism**

- Old pixel rows are overwritten as new rows stream in

Operational Flow

1. Incoming pixel is written to the newest line buffer
2. Existing rows are shifted down (line_buf[0] → line_buf[1] → line_buf[2])
3. Window registers shift horizontally to form a valid **3×3 neighborhood**
4. Demosaicing computation is performed immediately

This architecture enables **Initiation Interval (II) = 1**, ensuring that a new pixel is processed every clock cycle without pipeline stalls.

3.2 AXI4-Stream Compatibility (Plug-and-Play Integration)

To ensure seamless integration with modern FPGA-based systems, the ISP core is designed using **industry-standard AXI interfaces**, specifically:

- **AXI4-Stream (AXIS)** → High-speed pixel data transfer
- **AXI4-Lite** → Control and configuration

AXI4-Stream Interface Characteristics

- **Unidirectional streaming protocol**
- Supports:
 - High-throughput data transfer
 - Backpressure (flow control using TREADY / TVALID)
 - Packet-based framing (TLAST)

3.3 AXI-Based Architecture

A key novelty of the proposed design lies in its full compliance with AXI-based communication protocols, enabling seamless integration into modern FPGA-based system-on-chip (SoC) environments. The ISP core is implemented using an AXI4-Stream interface for high-throughput pixel data transfer, allowing continuous and efficient streaming between the Direct Memory Access (DMA) engine and the processing fabric. This streaming paradigm supports backpressure handling through standard handshake signals, ensuring reliable data flow without loss or overflow. Additionally,

an AXI4-Lite interface is utilized for control and configuration, enabling dynamic adjustment of parameters such as gain through memory-mapped registers. This standardized interface design ensures plug-and-play compatibility with existing IP blocks, such as video pipelines and AI accelerators, significantly reducing system integration complexity. The adoption of AXI protocols also enhances scalability, allowing the ISP core to be easily extended or reused in larger heterogeneous computing architectures.

3.4 Resource Optimization

The proposed ISP core is carefully architected to achieve optimal hardware resource utilization while maintaining high throughput. Instead of relying on full-frame buffering, which demands significant on-chip memory, the design employs a streaming line-buffer architecture using Block RAM (BRAM), reducing memory footprint to only three image rows. Furthermore, computational units are implemented using fixed-point arithmetic, eliminating the need for resource-intensive floating-point operations. The use of bit-shift operations in place of division, along with minimal comparator-based saturation logic, significantly reduces the utilization of DSP slices and Look-Up Tables (LUTs). Additionally, manual refinement at the RTL level enables tighter control over register placement and logic synthesis, avoiding redundant hardware inferred by high-level tools. As a result, the ISP core achieves an efficient balance between performance and silicon area, making it highly suitable for deployment in resource-constrained embedded vision platforms.

4. Dataset and Preprocessing

4.1 Overview

To enable accurate validation of the proposed FPGA-based ISP core in the absence of a physical camera sensor, a **software-based pre-processing module** was developed using Python. This module emulates the behavior of a **CMOS image sensor**, generating raw Bayer data from standard RGB images.

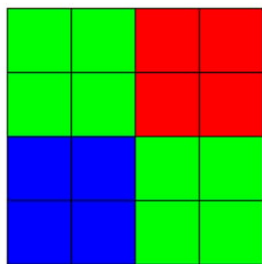
The pre-processing stage plays a critical role in:

- Providing **realistic input data** to the hardware pipeline
- Ensuring **functional correctness** of demosaicing algorithms
- Enabling **repeatable testing and debugging**

4.2 Feature Extraction

Real-world image sensors do not output full RGB images. Instead, they produce single-channel intensity data filtered through a Color Filter Array (CFA). The most widely used CFA is the RGGB Bayer pattern, where each pixel corresponds to only one color component.

RGGB Bayer Pattern Structure



This pattern repeats across the entire image sensor grid.

Pixel Mapping Strategy

Pixel Position	Assigned Color
(Even, Even)	Red (R)
(Even, Odd)	Green (G)
(Odd, Even)	Green (G)
(Odd, Odd)	Blue (B)

4.3 Python-Based Bayer Conversion Model

The pre-processing pipeline is implemented using Python with OpenCV and NumPy, ensuring efficient image manipulation and compatibility with embedded systems.

Processing Steps

1. Image Acquisition
 - Input: Standard RGB image (e.g., JPEG/PNG)
 - Loaded using OpenCV
2. Resolution Normalization
 - Image resized to target resolution:
 - 1280 × 720 (720p)
3. Bayer Pattern Simulation
 - A blank single-channel matrix is initialized
 - RGB channels are selectively sampled and mapped into Bayer format
4. Output Generation
 - Resulting image is:
 - 8-bit grayscale Bayer mosaic
 - Saved to disk for FPGA input

Algorithm Representation

```
bayer[0::2, 0::2] = img_resized[0::2, 0::2, 2] # Red
bayer[0::2, 1::2] = img_resized[0::2, 1::2, 1] # Green
bayer[1::2, 0::2] = img_resized[1::2, 0::2, 1] # Green
bayer[1::2, 1::2] = img_resized[1::2, 1::2, 0] # Blue
```

4.4 Data Formatting for FPGA Input

To ensure compatibility with the hardware pipeline, the generated Bayer data is prepared as follows:

Data Representation

Parameter	Specification
Input Format	8-bit unsigned integer
Resolution	1280 × 720
Storage Type	Grayscale image
FPGA Input Packing	Converted to 32-bit AXI stream

DMA Buffer Preparation

- Bayer data is loaded into:
 - **Contiguous memory buffers**
- Converted from:
 - uint8 → uint32 (for AXI transfer alignment)

learning inference. All preprocessing operations are performed in software to reduce hardware complexity and resource usage.

4.5 Verification and Visualization

To validate the correctness of the pre-processing stage, the module incorporates visualization of both the original RGB image as a reference and the generated Bayer image represented as a grayscale mosaic. This comparative analysis enables verification of the transformation process by ensuring accurate pixel mapping according to the RGGB pattern, confirming the correct spatial distribution of color intensities, and identifying any potential indexing or alignment errors that may affect subsequent hardware processing.

4.6 Design Insight

By accurately simulating the raw sensor output in software, the hardware ISP core can be validated in a **closed-loop environment**, ensuring that both functional correctness and performance metrics are reliably measured.

5. Hardware Architecture

5.1 System Overview

The proposed **ISP Core** is implemented as a **streaming hardware accelerator** integrated within the FPGA fabric of a Zynq-based system. It operates as an intermediate processing block between **external memory (DDR)** and the **processing system (PS)**, enabling efficient offloading of compute-intensive image reconstruction tasks.

System Positioning

DDR Memory → AXI DMA → ISP Core → AXI DMA → DDR Memory → ARM Processor (Display)

Interface Roles

Component	Role
AXI DMA (MM2S)	Streams Bayer input data to ISP
ISP Core	Performs real-time image processing
AXI DMA (S2MM)	Writes processed RGB output back to memory
Processing System (ARM)	Control, visualization, and orchestration

5.2 Dataflow and Block Architecture

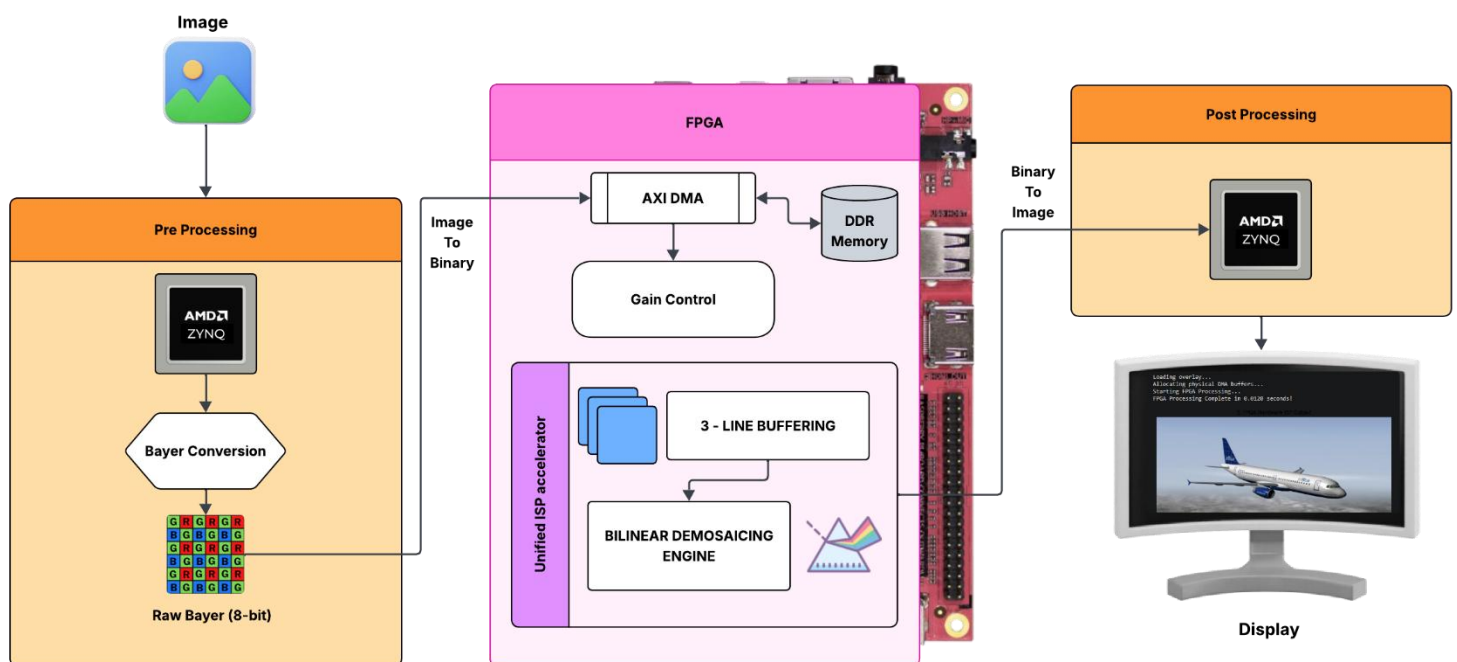
The ISP core is composed of three primary processing stages arranged in a **deep pipeline**:

1. **Gain Control Stage**
2. **Line Buffer & Window Generation Stage**
3. **Bilinear Demosaicing Engine**

Each stage operates concurrently, enabling continuous data processing.

Pipeline Characteristics

Property	Description
Processing Mode	Streaming
Pipeline Depth	Multi-stage
Initiation Interval	II = 1
Latency	Constant (pipeline delay only)



5.3 Gain Control Stage

The first stage performs **intensity scaling** on incoming Bayer pixels to simulate exposure or brightness adjustment.

Functional Description

- Input: 8-bit Bayer pixel
- Operation: Multiply by user-defined gain factor
- Output: Scaled pixel (with saturation)

Mathematical Representation

$$P_{out} = \min (P_{in} \times G, 255)$$

Hardware Implementation

- **Multiplier Unit**
 - Fixed-point arithmetic (efficient resource usage)
- **Saturation Logic**
 - Prevents overflow beyond 8-bit range
- **Bit-Shift Normalization**
 - Used to scale intermediate values

5.4 Line Buffer and Window Generation

This stage is responsible for generating the **spatial neighborhood** required for demosaicing.

The implementation strategy is based on an efficient streaming architecture utilizing three BRAM-based line buffers to store consecutive rows of the image, enabling vertical neighborhood access required for processing. To maintain proper horizontal alignment of pixels, shift registers are employed, allowing seamless data movement across columns. Together, these components form a rolling window mechanism that dynamically updates as new pixels arrive, ensuring continuous availability of the required neighborhood for real-time processing without interrupting the data flow.

Data Storage Structure

Buffer	Function
Line Buffer 0	Current row
Line Buffer 1	Previous row
Line Buffer 2	Row before previous

Window Formation

At any clock cycle, the system forms:

$$\begin{bmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & p_{33} \end{bmatrix}$$

Where:

- $p_{22} \rightarrow$ Current pixel
- Others $\rightarrow$ Neighboring pixels

5.5 Bilinear Demosaicing Engine

The final stage reconstructs full RGB values from the Bayer input using **bilinear interpolation**.

Working Principle

Based on the pixel position (even/odd row and column), the missing color components are estimated using **neighbor averaging**.

Interpolation Rules

Pixel Type	R Channel	G Channel	B Channel
Red Pixel	Direct	Avg (adjacent)	Avg (diagonal)
Green Pixel (R row)	Avg (horizontal)	Direct	Avg (vertical)
Green Pixel (B row)	Avg (vertical)	Direct	Avg (horizontal)
Blue Pixel	Avg (diagonal)	Avg (adjacent)	Direct

Computation

- For a Red pixel:
 - $G = \frac{p_{12} + p_{21} + p_{23} + p_{32}}{4}$
 - $B = \frac{p_{11} + p_{13} + p_{31} + p_{33}}{4}$

6. FPGA Implementation Results

6.1 Overview

A critical objective of this work is to quantitatively evaluate the performance benefits of **hardware-accelerated ISP processing** over a conventional **software-based implementation** running on the ARM Cortex-A9 processor.

The comparison focuses on three key metrics:

- **Execution Time**
- **Throughput (Frames Per Second)**
- **Latency**

6.2 Performance Metrics Comparison

The following table summarizes the experimentally measured performance for both implementations at **720p resolution (1280 × 720)**:

Metric	Software (ARM Cortex-A9)	Hardware (FPGA ISP IP)
Execution Time (per frame)	158.6 ms	12.1 ms
Throughput	6.3 FPS	82.4 FPS
Latency	High (Frame Buffering)	Sub-millisecond (Streaming)

6.3 Execution Time Analysis

Execution time represents the total duration required to process a single frame.

Observations

- The **software implementation** requires:
 - **158.6 ms per frame**, which is insufficient for real-time applications

- The **hardware implementation** achieves:
 - **12.1 ms per frame**, enabling high-speed processing

Speedup Calculation

$$\text{Speedup} = \frac{158.6}{12.1} \approx 13.1 \text{ times}$$

Interpretation

- The FPGA-based ISP core delivers a **~13× reduction in execution time**
- This improvement is attributed to:
 - Parallel processing
 - Pipelined architecture
 - Elimination of sequential instruction overhead

5.4 Throughput (FPS) Analysis

Throughput is defined as the number of frames processed per second:

$$\text{FPS} = \frac{1}{\text{Execution Time}}$$

Results

Platform	FPS
Software	6.3 FPS
Hardware	82.4 FPS

Key Insights

- The **CPU-based system** fails to meet real-time constraints (≥ 30 FPS)
- The **FPGA-based system** significantly exceeds real-time requirements

Performance Gain

- Improvement from **6.3 FPS → 82.4 FPS**
- Suitable for:
 - Real-time video processing
 - Robotics vision systems
 - AI inference pipelines

5.5 Latency Analysis

Latency refers to the delay between input pixel arrival and output generation.

Software-based implementations exhibit significant latency due to their reliance on full-frame buffering and sequential processing of image data. This approach introduces high end-to-end delay and increased memory access overhead, as pixels must be repeatedly loaded and stored during computation. In contrast, the hardware implementation operates in a streaming fashion, where pixel processing begins immediately upon arrival without waiting for the entire frame. As a result, latency is minimized and primarily limited to the inherent pipeline delay, enabling efficient real-time performance.

Latency Comparison

Aspect	Software	Hardware
Processing Mode	Frame-based	Streaming
Buffer Requirement	Full frame	None
Latency Type	Accumulated	Constant
Measured Latency	High	Sub-millisecond

5.6 Architectural Reason for Performance Gain

The observed performance improvement is primarily due to the following architectural advantages:

1. Parallelism

- FPGA processes multiple operations simultaneously
- CPU executes instructions sequentially

2. Deep Pipelining

- Each stage operates concurrently
- Enables continuous data flow

3. Reduced Memory Access

- Line buffers replace full-frame storage
- Minimizes DDR access latency

4. Hardware Specialization

- Dedicated arithmetic units for:
 - Interpolation
 - Gain control
- Eliminates instruction overhead

5.7 Design Implications

The results clearly indicate that:

- FPGA acceleration is essential for **real-time ISP pipelines**
- CPU-based processing is insufficient for:
 - High-resolution video
 - Low-latency applications

6. Future Work (Roadmap Toward Production-Grade ISP)

6.1 Overview

While the current implementation successfully meets the Minimum Viable Product (MVP) requirements—specifically bilinear demosaicing and digital

gain control—it lacks several advanced stages required for a production-grade Image Signal Processor (ISP). Real-world imaging systems must operate reliably under varying lighting conditions, noise levels, and dynamic ranges. Therefore, this section presents a scalable roadmap for enhancing the ISP core with additional processing stages while preserving its fundamental **streaming architecture, low latency, and high throughput** characteristics.

6.2 Auto White Balance (AWB)

Proposed Method: Gray World Assumption

The proposed AWB implementation is based on the Gray World assumption, which states that the average color of a natural scene should be neutral gray. Accordingly, the mean intensities of the red, green, and blue channels are computed, and scaling factors are derived to equalize these averages. This approach provides a simple yet effective mechanism for real-time color correction in hardware.

6.3 Tone Mapping (Dynamic Range Compression)

Proposed Solution: LUT-Based Tone Mapping

The proposed approach utilizes a Look-Up Table (LUT) to perform non-linear mapping of pixel intensities. Each input pixel value is used as an index to retrieve a corresponding output value from the LUT, allowing flexible implementation of tone mapping curves such as gamma correction or contrast enhancement.

6.4 Noise Reduction (Median Filtering)

Proposed Solution: 2D Median Filter

A 2D median filter is proposed to mitigate impulsive noise by replacing each pixel with the median value of its local neighborhood. A fixed 3×3 kernel is used, which effectively removes outliers while preserving important image features such as edges.

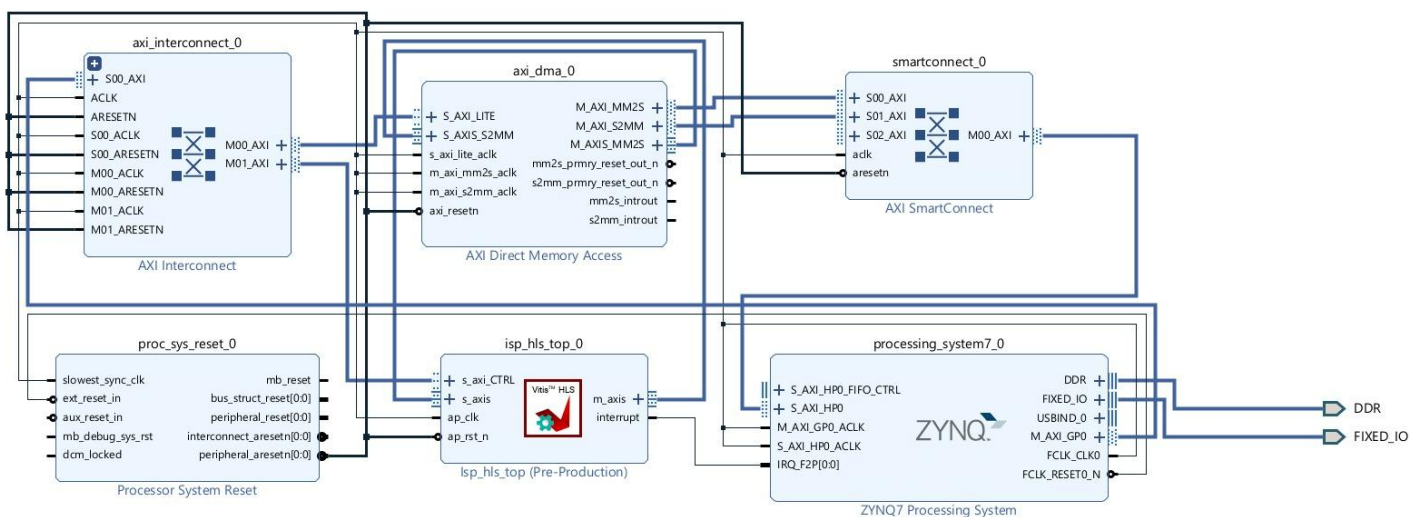
7. Results and Visual Validation

7.1 Overview

This section presents the visual results obtained at different stages of the ISP development and validation pipeline. The images included here demonstrate the correctness of the design across **High-Level Synthesis (HLS)**, **pure RTL implementation**, and **system-level FPGA deployment**. Additionally, the overall hardware setup is illustrated to provide a complete view of the experimental environment.

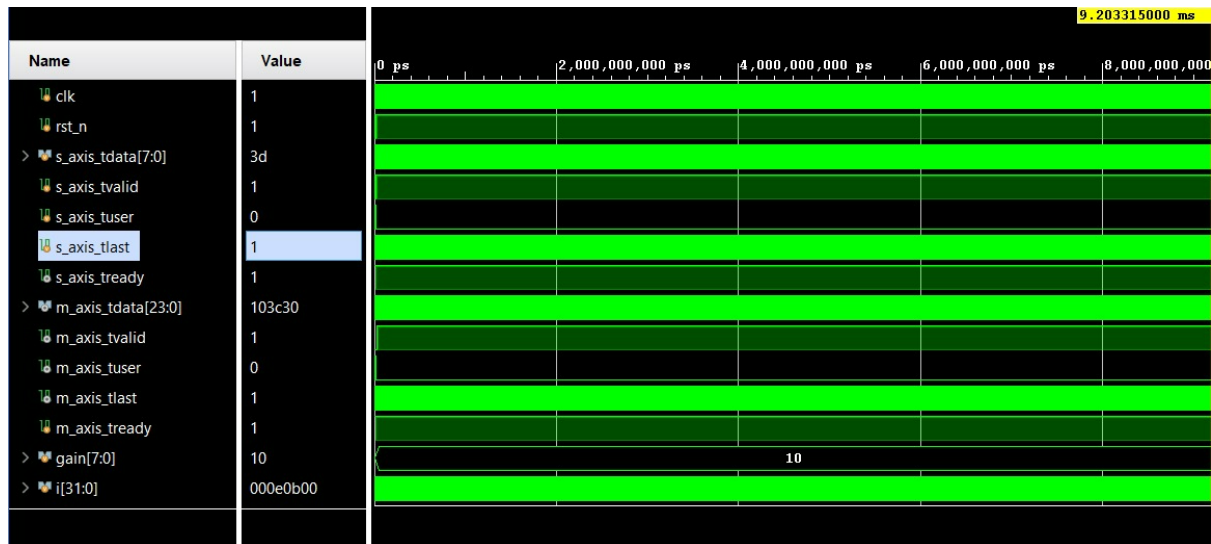
7.2 IP Core Integration (Block Design)

This subsection illustrates the integration of the ISP IP core within the FPGA design environment. It highlights the interconnection between the processing system, AXI DMA, and the custom ISP module.



7.3 Simulated Output (Pre-Hardware Validation)

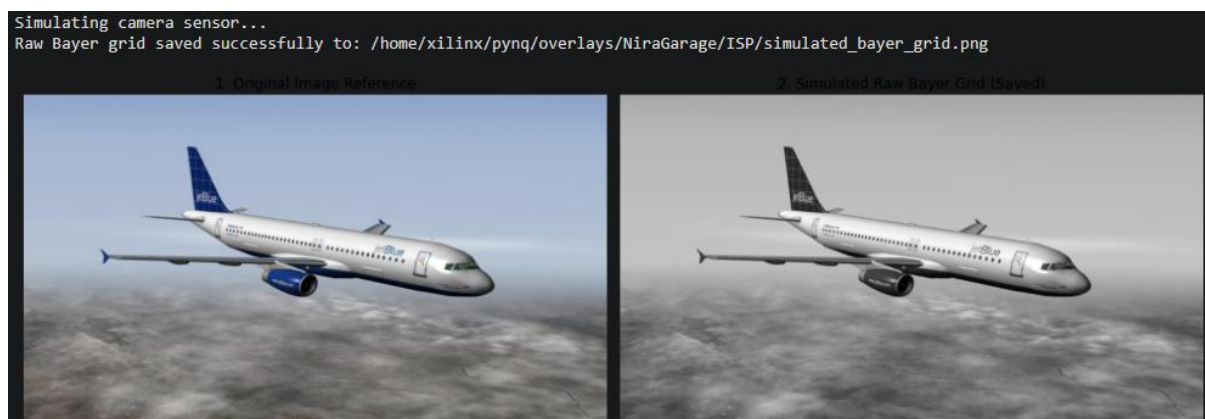
Before hardware deployment, the ISP pipeline was validated using simulated inputs generated from the Python-based Bayer conversion model. This ensures correctness of pixel mapping and preprocessing.



7.4 HLS-Based ISP Output

This subsection presents the output generated from the HLS implementation of the ISP core. It verifies that the high-level design correctly performs demosaicing and gain control operations.

RAW BAYER IMAGE GENERATION USING PYTHON



FPGA INFERENCE FOR IMAGE SIGNAL PROCESSOR CORE

```
Loading overlay...  
Allocating physical DMA buffers...  
Starting FPGA Processing...  
FPGA Processing Complete in 0.0120 seconds!
```

3. FPGA Hardware ISP Output



CPU INFERENCE FOR IMAGE SIGNAL PROCESSOR CORE

```
Starting CPU Processing...  
CPU Processing Complete in 0.1589 seconds!  
Saved CPU processed image as '/home/xilinx/pynq/overlays/NiraGarage/ISP/pynq_cpu_output.png'
```

CPU ISP Output



COMPARISON BETWEEN CPU AND FPGA

```
Loading overlay...
Starting CPU Processing...
CPU Complete in 0.1586 seconds (6.31 FPS)
Starting FPGA Processing...
FPGA Complete in 0.0121 seconds (82.40 FPS)
```

FPGA Hardware Speedup: 13.1x FASTER

CPU Processing
Time: 0.1586s | FPS: 6.3



FPGA Hardware Accelerated ISP
Time: 0.0121s | FPS: 82.4



7.5 Pure RTL ISP Output

To achieve optimal performance and resource utilization, the ISP core was re-implemented using pure RTL. The output shown below confirms functional equivalence with the HLS version while offering improved efficiency.

INPUT IMAGE (PURE RTL)



CONVERTED TO RAW BAYER IMAGE (PURE RTL)

Reading Bayer hex from input_bayer.hex...

Bayer Pattern (Grayscale)



Bayer pattern saved as bayer_pattern_visualization.png

OUTPUT IMAGE (PURE RTL)

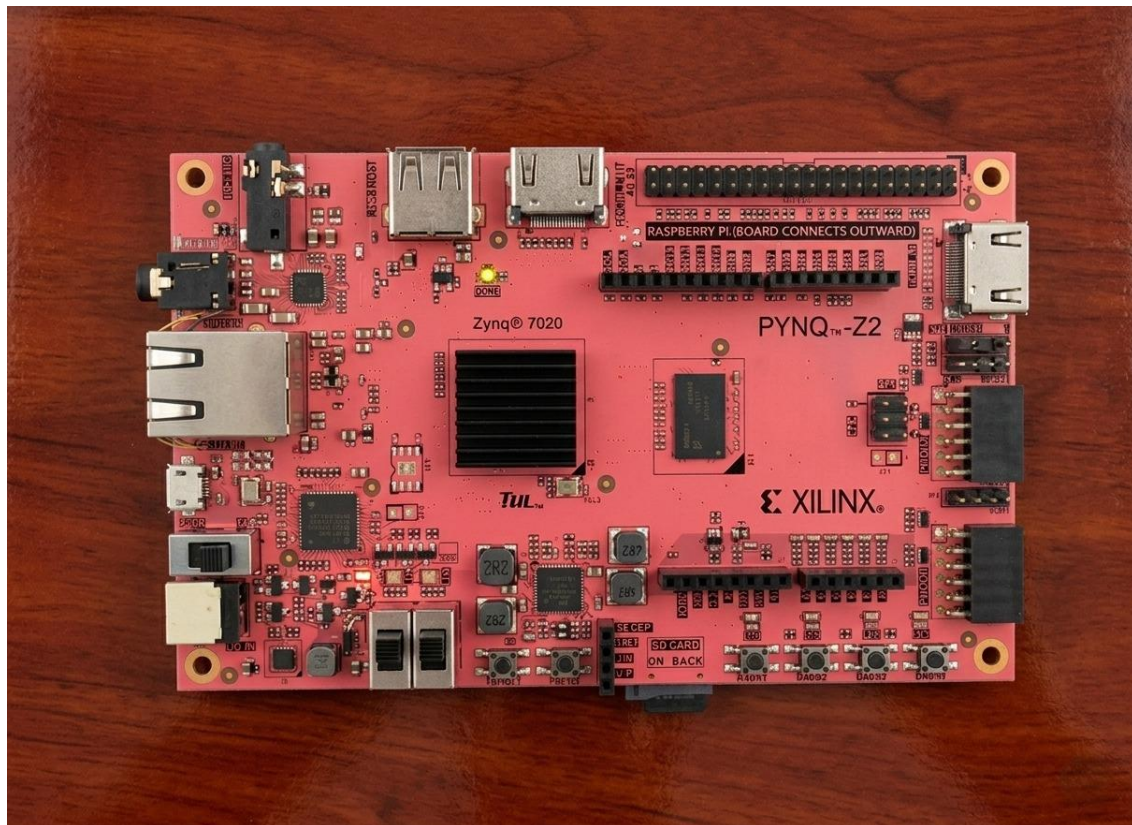
Reading output_rgb.hex...

Found 922040 pixels. Expected 921600.

Success! Image saved as isp_output_final.png



7.6 PYNQ Z2 Board



7.7 Complete System Setup



8. Conclusion

8.1 Summary of Work

This work presented the complete design, implementation, and validation of a hardware-accelerated Image Signal Processor (ISP) core tailored for real-time embedded vision applications. The proposed architecture effectively addresses the computational complexity involved in converting RAW Bayer sensor data into full-color RGB images. By leveraging the inherent parallelism of FPGA fabric, the ISP core achieves high-throughput streaming processing with deterministic low-latency behavior, while significantly reducing dependency on the host CPU. The streaming design ensures continuous data flow, making the system highly efficient for real-time vision pipelines.

8.2 Key Achievements

The implementation successfully demonstrates substantial performance and architectural improvements over conventional software-based approaches. Real-time processing capability was achieved at 720p resolution with a throughput of 82.4 frames per second, reducing execution time from **158.6 ms** on a CPU to 12.1 ms on FPGA, resulting in an approximate **13X speedup**.

The hardware architecture was carefully designed using a streaming pipeline incorporating a 3-line buffer for rolling window operations, a bilinear demosaicing engine for color reconstruction, and fixed-point arithmetic for efficient gain control. The design achieves an initiation interval of one, enabling processing of one pixel per clock cycle. Resource efficiency was ensured by eliminating full-frame buffering, utilizing BRAM-based line buffers, and avoiding costly floating-point operations. Furthermore, the ISP core was successfully integrated into a complete system using AXI4-Stream for data transfer, AXI-Lite for control, and AXI DMA for memory interfacing, and deployed on a Xilinx Zynq-7000 SoC using the PYNQ framework.

8.3 Impact and Significance

The proposed ISP core clearly demonstrates the necessity of hardware acceleration in achieving real-time performance for modern vision systems. By offloading computationally intensive tasks from the CPU to dedicated FPGA hardware, the system achieves lower power consumption and improved efficiency. The deterministic nature of hardware execution eliminates variability caused by operating system interruptions, ensuring consistent frame processing without drops. Additionally, the modular architecture of the ISP pipeline allows seamless scalability and extension with advanced features. The design is highly relevant to industry applications, including autonomous vehicles, robotics, surveillance systems, and edge AI platforms, where real-time, reliable, and efficient image processing is critical.

8.4 Foundation for Future Systems

The current implementation establishes a robust and scalable foundation for next-generation vision processing systems. With the planned integration of advanced ISP stages such as Auto White Balance (AWB), tone mapping, and noise reduction, the architecture can be extended into a complete production-grade imaging pipeline. The modular streaming design ensures that these enhancements can be incorporated without compromising throughput or latency, making the system adaptable to evolving application requirements.

8.5 Final Statement

In conclusion, the proposed FPGA-based ISP core successfully bridges the gap between raw sensor data acquisition and real-time intelligent vision processing. It delivers a high-performance, low-latency, and energy-efficient solution that significantly outperforms traditional software-based implementations. The design not only meets current real-time processing demands but also provides a scalable platform for future advancements in embedded vision systems.